دليلك الشامل لرفع موقعك على VPS

من كتابة الكود إلى إطلاق الموقع باستخدام النسخة الأحدث من Docker

---

مقدمة: ليه تقرأ الكتاب ده؟

لو إنت بتعمل مواقع أو تطبيقات، أكيد جربت ترفع شغلك على استضافات عادية (Shared Hosting) وحسيت إنها بطيئة أو مقيدة، أو جربت خدمات سحابية ولقيت الفاتورة بتسحب منك أرقام خيالية. هنا بييجي دور الـ VPS (Virtual Private Server).

ببساطة، الـ VPS (Virtual Machine) هو سيرفر افتراضي داخل سيرفر أكبر حقيقي باستخدام تقنية (Hypervisor). بيوفر لك بيئة معزولة تماماً كأنك تملك السيرفر لوحدك، شغال 24 ساعة، وبيديك تحكم كامل (Root Access). في الكتاب ده، هنمشي خطوة بخطوة عشان ترفع أي بروجيكت مهما كان حجمه (سواء كان بـ Node.js، React، PHP، Python أو غيره) على سيرفرك الخاص، وبأحدث الطرق الاحترافية.

ماذا ستحتاج للبدء؟

قبل الدخول في الخطوات العملية، يُفضل أن يكون لديك:

- **حساب GitHub:** لرفع كود مشروعك عليه.
- **مشروع برمجي جاهز:** المشروع الذي ترغب في رفعه على السيرفر.
- **اسم نطاق (Domain):** لربطه بموقعك لاحقاً (مثل `yourdomain.com`).

الفصل الأول: البداية وتجهيز السيرفر

أول حاجة محتاجينها هي إننا نأجر السيرفر. في شركات كتير ممتازة وأسعارها رخيصة بتبدأ من 4 أو 5 دولار في الشهر، وأشهرها **DigitalOcean**، أو **Hetzner** (ممتازة تقنياً)، أو **Hostinger**، أو **Contabo**.

إيه الفرق بين خطط الـ KVM (من KVM 1 لـ KVM 8)؟

الـ KVM هو اختصار لـ *Kernel-based Virtual Machine*، وهو نوع التقنية المستخدمة لعزل النظام الخاص بك تماماً، وإن كانت الموارد قد تكون مشتركة في بعض الشركات.

- **خطة KVM 1:** (1 vCPU، 4GB RAM، 50GB مساحة) – مثالية للمبتدئين والمواقع الصغيرة.
- **خطة KVM 2:** (2 vCPU، 8GB RAM، 100GB مساحة) – للمتاجر الإلكترونية الصغيرة والتطبيقات المتوسطة.
- **خطة KVM 4:** (4 vCPU، 16GB RAM، 200GB مساحة) – للشركات والمنصات التعليمية.
- **خطة KVM 8 (فأعلى):** (8 vCPU، 32GB RAM، 400GB مساحة) – للمشاريع الضخمة Enterprise Level.

الخلاصة للبدء: ابدأ دائماً بخطة **KVM 1** أو **KVM 2**. ميزة الـ VPS إنك تقدر تعمل Upgrade في أي لحظة بدون ما موقعك يقع.

خطوات حجز السيرفر:

1. بتعمل حساب على أي شركة استضافة (مثل DigitalOcean أو Hetzner أو Hostinger).
2. بتختار قسم الـ VPS Hosting وتختار الخطة المناسبة.
3. بتختار نظام التشغيل **Ubuntu** (الإصدار 22.04 أو الأحدث).

4. بعد الدفع، الشركة هتديك رقم الـ IP، اسم المستخدم `root` ، والباسورد.

إزاي تدخل على السيرفر بتاعك؟

```
ssh root@your_server_ip
```

أول وأهم خطوة (تحديث النظام): بمجرد الدخول للسيرفر لأول مرة، دائماً حدث الحزم أولاً:

```
apt update && apt upgrade -y
```

الفصل الثاني: سر المحترفين (تسطيب Docker الحديث)

الدوكر بياخد البروجيكت بتاعك بكل اللي بيحتاجه، ويحطه في "صندوق" مقفول (Container) وشغال لوحده. عشان نشتغل باحترافية، هنسطب النسخة الرسمية الأحدث من Docker.

1. تحديث النظام وتنزيل الأدوات الأساسية:

```
apt-get update
apt-get install ca-certificates curl gnupg
```

2. إضافة مفتاح الأمان الخاص بـ Docker:

```
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg -
-dearmor -o /etc/apt/keyrings/docker.gpg
chmod a+r /etc/apt/keyrings/docker.gpg
```

3. إضافة مستودع Docker الرسمي للنظام:

```
echo \
  "deb [arch="$(dpkg --print-architecture)" signed-
by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \
  "$(. /etc/os-release && echo "$VERSION_CODENAME")" stable" |
\
  tee /etc/apt/sources.list.d/docker.list > /dev/null
```

4. تسطيب أحدث إصدار من Docker و Docker Compose V2:

```
apt-get update
apt-get install docker-ce docker-ce-cli containerd.io docker-
buildx-plugin docker-compose-plugin
```

الفصل الثالث: التطبيق العملي ورفع الكود

1. سحب الكود للسيرفر:

```
git clone https://github.com/yourusername/your-repo-name.git
```

2. الدخول لمجلد المشروع:

```
cd your-repo-name
```

3. تشغيل المشروع (مثال لملف `docker-compose.yml` مناسب لـ Production):

```
services:
  web:
    build: .
    restart: unless-stopped
    ports:
      - "127.0.0.1:3000:3000" # مهم: ليمر عبر Nginx فقط
    # ... باقي الإعدادات
```

انتبه لثغرة خطيرة: استخدام `"3000:3000"` بالشكل التقليدي يجعل موقعك متاحاً مباشرة عبر `http://ip:3000` متجاوزاً Nginx والـ Firewall! استخدم دائماً `127.0.0.1:3000:3000`.

```
docker compose up -d
```

الفصل الرابع: ربط الدومين وتأمين الموقع

1. ربط الدومين: أضف A Record في إعدادات الـ DNS للدومين بتاعك يشاور على الـ IP بتاع السيرفر.

ملاحظة: انتشار تحديثات الـ DNS قد يستغرق من بضع دقائق إلى 24 ساعة.

2. تسطيب Nginx:

```
apt-get install nginx
```

3. إعداد Nginx لموقعك:

```
nano /etc/nginx/sites-available/yourdomain.com
```

```
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
ln -s /etc/nginx/sites-available/yourdomain.com
/etc/nginx/sites-enabled/
systemctl restart nginx
```

4. تأمين الموقع بـ SSL مجاني:

```
apt-get install certbot python3-certbot-nginx
certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

الفصل الخامس: تحديث الموقع بعد أي تعديل

على جهازك:

```
git add .
git commit -m "update features"
git push
```

على السيرفر:

```
cd your-repo-name
git pull
docker compose up -d --build
```

الفصل السادس: استضافة أكثر من موقع على نفس الـ VPS

لكل موقع إضافي: غير البورت الخارجي في `docker-compose.yml` (مثلاً `127.0.0.1:4000:3000`)، أضف A Record للدومين الجديد، وأنشئ ملف Nginx منفصل له.

الفصل السابع: التأمين الجذري للسيرفر (Security Hardening)

1. تفعيل الجدار الناري UFW:

```
apt install ufw
ufw allow OpenSSH
ufw allow 'Nginx Full'
ufw enable
```

2. حماية الـ SSH وتثبيت Fail2ban: يُنصح بإعداد SSH Keys وتعطيل دخول الـ root بكلمة مرور من ملف `/etc/ssh/sshd_config`، ثم:

```
apt install fail2ban -y
systemctl enable fail2ban
systemctl start fail2ban
```

الفصل الثامن: ترقية بيئة العمل (CI/CD Pipelines)

الأفضل من الـ `git pull` اليدوي هو استخدام **GitHub Actions** أو **GitLab CI**، اللي بينتج عنه كل `git push` من جهازك يؤدي تلقائياً لتحديث السيرفر. ابحث عن (GitHub Actions Docker VPS) لتتعلم كيفية تطبيقها.

الفصل التاسع: النسخ الاحتياطي والمراقبة

الباك-أب: استخدم `rsync` مع `cron` لأخذ نسخة يومية من قاعدة البيانات وملفاتك ونقلها لـ Object Storage.

المراقبة: ثبت **Prometheus & Grafana** لرسم إحصائيات استهلاك الـ CPU والرامات لحظة بلحظة، وإرسال تنبيهات عند الضغط الزائد.

الخاتمة

مبروك! إنت دلوقتي مش مجرد شخص بيكتب كود، إنت بقيت بتعرف تعمل Deployment باحترافية وتدير السيرفر بتاعك زي مهندسين الـ DevOps. استخدامك للـ VPS مع أحدث إصدارات Docker و Nginx هيضمنلك سرعة، أمان، واستقرار لمشاريعك.